MODELENGINE OPEN SOURCE PROJECT COMPETITION

ModelEngineBySoldierBoy

医疗数据智能体系统项目书

基于 DataMate 与 Nexent 构建“数据处理 → 医疗知识图谱 → 图谱驱动分析与可视化”的完整智能体链路

项目基本信息

项目方向	医疗数据处理、知识图谱生成、数据分析与可视化智能体系统
代码仓库	https://gitlink.org.cn/ohanabi/ModelEngineBySoldierBoy.git
提交分支	progress/competition-agent-chain
文档日期	2026-07-06
交付形态	CLI / HTTP API / Nexent OpenAPI-MCP 工具 / Dashboard / PDF 项目书

文档	实体	三元组	NL2SQL
320	4468	3403	70/70

本项目同时包含任务一可运行 Demo、任务二医疗知识图谱、任务三图谱驱动 BI 展示、小于 1B 的 LoRA 编排策略模型，以及 Ascend NPU 算子优化实现。

目录

1 项目总体介绍 3

2 项目场景与要解决的问题 3

3 系统总体架构 4

4 核心工程模块 5

5 可运行 Demo 与系统交付形态 6

6 DataMate 与 Nexent 集成设计 7

6.1 DataMate 在系统中的位置 7

6.2 Nexent 在系统中的位置 7

7 任务一：数据处理智能体详细设计 7

7.1 输入输出与可运行 Demo 7

7.2 任务理解与语义约束图 8

7.3 多目标 DAG 规划 8

8 任务二：医疗知识图谱智能体详细设计 9

8.1 医疗数据源 9

8.2 图谱 schema 9

8.3 核心算子 9

9 任务三：数据分析智能体与可视化 10

9.1 图谱感知与 NL2SQL 10

9.2 BI 与图文展示 10

10 模型侧增强：Qwen3-0.6B + LoRA 编排策略模型 10

10.1 为什么需要模型侧增强 11

10.2 训练数据与训练方式 11

10.3 接入主链路 11

11 NPU 算子优化重点说明 12

11.1 任务一 NPU Ranker 12

11.2 任务二 NPU KG Tensor Kernel 12

11.3 任务三 NPU 图谱相似度算子 13

12 运行结果、交付方式与复现 13

12.1 可运行入口 13

12.2 交付文件 14

13 项目总结 14

1 项目总体介绍

ModelEngineBySoldierBoy 是一个面向医疗数据场景的多智能体系统。项目围绕“数据 → 知识 → 洞察”的完整链路建设，将医疗原始数据处理、医疗知识图谱生成、图谱问答、数据分析、NL2SQL 和 BI 可视化整合为一个可运行的系统。

项目的核心目标不是只做一个固定样例流程，而是构建一个能够理解数据处理任务、自动规划处理步骤、编排多个算子、跟踪执行状态、处理异常，并将处理结果继续用于知识图谱和数据分析的智能体群。三个智能体分别负责数据处理、知识生成和数据分析，同时通过统一的数据结构和平台接口串联起来。

系统以 DataMate 和 Nexent 作为底座。DataMate 提供数据接入、数据集管理、文件处理、知识库和数据处理能力；Nexent 提供智能体运行入口、OpenAPI/MCP 工具注册、工具发现、工具绑定和自然语言调用能力。本项目在这两个平台能力之上，实现面向医疗数据的任务理解、处理规划、知识抽取、图谱校验、分析规划、模型侧策略增强和 NPU 算子优化。

智能体	核心职责	输入	输出
DataProcessingAgent	医疗数据接入、清洗、抽取、转换、质量检查、异常恢复	CSV、TXT、JSONL、自然语言任务	标准化文档、质量报告、处理 trace、KG 输入
MedicalKGAgent	医疗实体抽取、关系抽取、三元组生成与校验、图谱问答	医疗文本、任务一处理产物、CBLUE 转换数据	实体、三元组、图谱 schema、问答 trace
InsightAnalysisAgent	图谱理解、NL2SQL、统计分析、关联分析、图谱驱动可视化	任务一文档、任务二图谱、自然语言分析需求	分析报告、SQL 结果、BI 图表、关系子图、Dashboard

项目增强能力

为了增强智能体能力，项目实现了两类附加技术路线。第一类是模型侧增强：训练小于 1B 的 Qwen3-0.6B + LoRA 编排策略模型，用于生成任务一数据处理流程候选策略，并通过规则/约束规划器校验后接入主链路。第二类是 NPU 算子优化：将任务一算子候选排序、任务二 KG candidate ranking + top-k kernel、任务三图谱多关系相似度计算改造成 tensor 算子，并在 Ascend 910B2C 上完成 CPU/NPU 对比。

2 项目场景与要解决的问题

医疗数据处理链路通常包含多种异构数据，既有结构化表格，也有病历摘要、指南片段、医学实体关系标注等非结构化文本。单纯做一个抽取脚本无法解决完整链路中的问题，因为真实场景至少包含以下难点。

1. 输入数据形态不统一。CSV 表、JSONL 文档、普通 TXT 文本和外部数据集转换文件具有不同字段结构，处理前必须识别文件类型、字段分布、缺失情况、重复情况和文本质量。
2. 数据处理任务不是固定动作。用户可能要求“清洗医疗文本并抽取字段”，也可能要求“对结构化表

做 ETL 并校验字段”，还可能要求“批量处理多文件并保留可追踪状态”。这要求智能体具备任务理解和规划能力。

3. 知识图谱构建需要医疗语义约束。实体不能只有字符串抽取，还要区分疾病、症状、药物、检查、治疗、风险因素等类型；关系也需要区分诊断、治疗、并发、风险、预防、因果等语义。
4. 数据分析需要理解图谱结构。任务三不是简单展示三元组数量，而是要从图谱中执行统计、关联、趋势或中心性分析，并将自然语言问题转化为 SQL 或图谱查询。
5. 系统需要可运行和可扩展。项目不能只产出一个不可复用的 Notebook，而应提供 CLI、HTTP API、Nexent OpenAPI/MCP 工具、数据产物、图谱产物、分析产物和可复现脚本。

围绕这些问题，本项目采用“平台能力 + 领域算子 + 智能体规划 + 模型策略 + NPU 算子优化”的组合方案。

3 系统总体架构

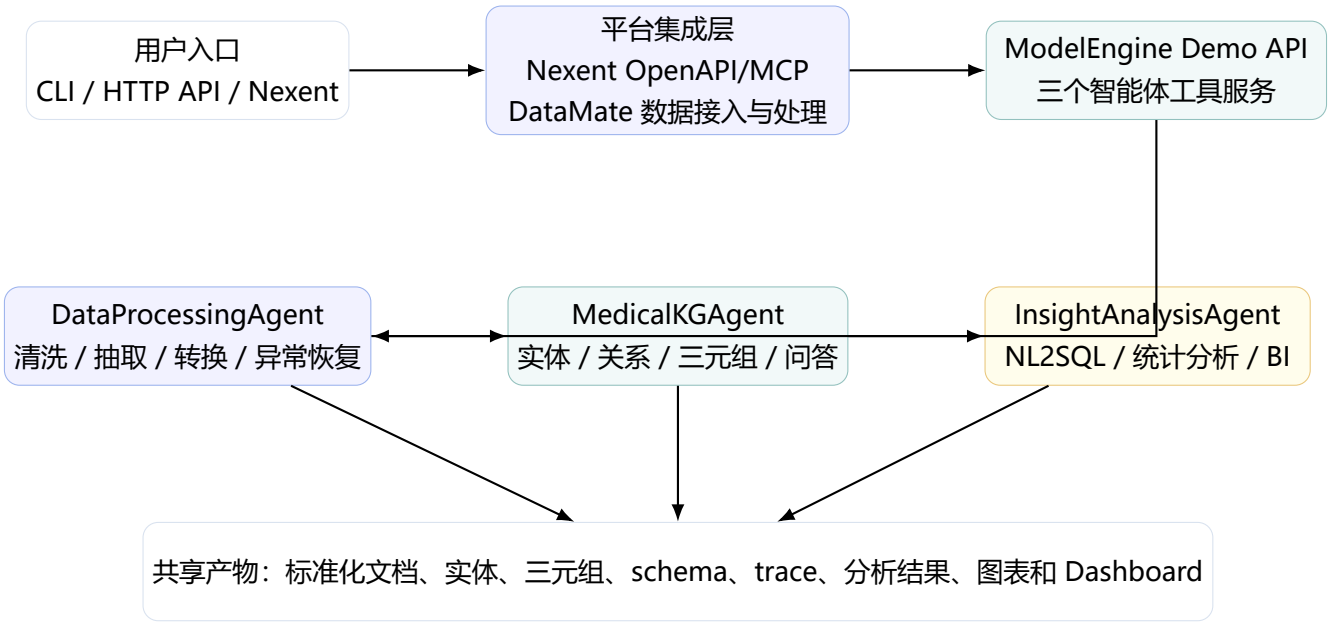


图 1: 系统总体架构。DataMate 和 Nexent 提供平台底座，三个智能体提供领域链路能力。

整体架构可以理解为六层：用户入口层、平台集成层、智能体编排层、算子与模型层、数据与图谱层、展示与交付层。Nexent 负责智能体工具注册与自然语言调用，DataMate 负责数据接入、文件处理、知识库和数据处理能力，项目本身负责医疗领域任务规划、知识抽取、图谱分析和可视化。

层级	组件	作用
用户入口层	CLI、HTTP API、Nexent 自然语言入口、Dashboard	接收任务、触发智能体、查看结果
平台集成层	DataMate、Nexent、ModelEngine Demo API	平台服务、工具注册、数据处理、API 入口

智能体编排层	DataProcessingAgent、MedicalKGAgent、InsightAnalysisAgent	理解任务、规划步骤、调度算子、处理异常
算子与模型层	医疗抽取算子、校验算子、NL2SQL、LoRA、NPU tensor 算子	执行具体计算与智能增强
数据与图谱层	标准化文档、实体、三元组、schema	跨任务复用的数据结构
展示与交付层	报告、trace、图表、子图、证据包	展示结果并支持复核

4 核心工程模块

项目代码按“智能体、算子、平台集成、数据产物、运行脚本”拆分。这样的组织方式保证三个任务边界清晰，算子可复用，中间产物可追踪。

模块	文件或目录	说明
数据处理智能体	agents/datamate_processing_agent.py	任务一核心智能体，负责任务理解、DataMate 数据接入、算子候选选择、DAG 规划、状态跟踪和异常处理
医疗知识图谱智能体	agents/medical_kg_agent.py	任务二核心智能体，负责编排实体抽取、关系抽取、三元组生成、校验、图谱导出和问答
数据分析智能体	agents/analysis_agent.py	任务三核心智能体，负责图谱理解、NL2SQL、统计分析、关联分析、图谱子图和可视化
平台集成智能体	agents/integration_agent.py	负责 DataMate/Nexent 配置、OpenAPI 注册、MCP 刷新、工具可见性和运行记录
Demo API	demo/app_server.py	HTTP 服务，也是 Nexent 调用三个智能体的工具服务
OpenAPI 描述	demo/task*_openapi.py	为 Nexent 提供工具接口描述
医疗抽取算子	operators/medical_pipeline.py	实体抽取、关系抽取、三元组生成主逻辑
KG 校验算子	operators/medical_kg_validation.py	类型约束、关系约束、冲突过滤、证据聚合和置信度校验
任务一 tensor ranker	operators/task1_tensor_ranker.py	对 DataMate operator market 候选进行 tensor 化排序
任务二 NPU tensor 算子	operators/npu_tensor_kg_ops.py	KG candidate ranking、top-k 和 tensor kernel

5 可运行 Demo 与系统交付形态

项目提供 CLI、HTTP API、Nexent OpenAPI/MCP 工具和 Dashboard 四种运行形态。CLI 用于快速复现三项任务；HTTP API 用于把智能体作为服务调用；Nexent OpenAPI/MCP 工具用于平台内自然语言触发；Dashboard 用于展示任务三图谱驱动分析结果和端到端产物。

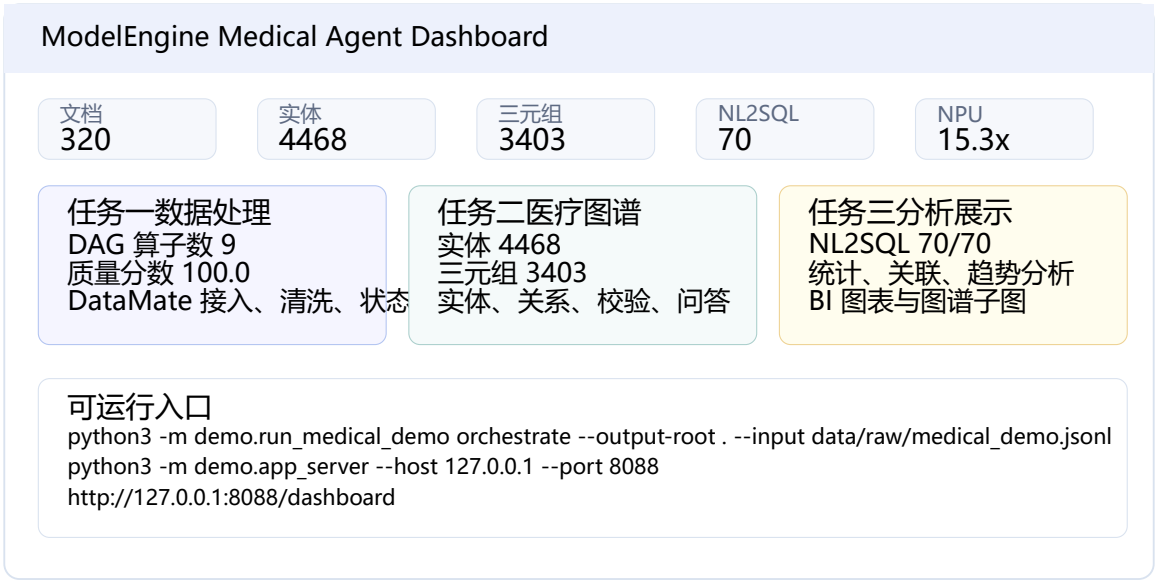


图 2: Dashboard 与 Demo 交付形态示意。实际页面由 demo/dashboard.html 提供，运行 API 服务后可访问。

完整链路运行命令如下。该命令会依次执行任务一数据处理、任务二医疗知识图谱构建和任务三分析展示，并生成 demo/orchestration_summary.json、demo/orchestration_trace.jsonl 和 demo/orchestration_repo

```
python3 -m demo.run_medical_demo orchestrate \
  --output-root . \
  --input data/raw/medical_demo.jsonl
```

HTTP API 服务启动后，三个任务均可以通过 REST 接口调用，同时这些接口也是 Nexent 工具注册使用的 OpenAPI 服务入口。

```
python3 -m demo.app_server --host 127.0.0.1 --port 8088

GET  /health
POST /api/task1/process
POST /api/task2/build
POST /api/task2/ask
POST /api/task3/analyze
GET  /dashboard
```

任务一可交付 Demo

任务一不是只作为内部前置步骤存在，而是有独立可运行入口、独立处理报告和独立平台工具。CLI 命令会输出处理计划、DAG、质量报告、trace 和标准化数据；HTTP API 可以被外部系统调用；Nexent 工具可以通过自然语言触发；Dashboard 和项目报告会展示任务一执行结果。

6 DataMate 与 Nexent 集成设计

6.1 DataMate 在系统中的位置

DataMate 是任务一数据接入与处理链路的核心。DataProcessingAgent 会围绕 DataMate API 完成数据处理流程。DataMate 在项目中承担以下角色：

- 数据集管理：创建或复用数据集，作为结构化和非结构化数据的管理入口。
- 文件接入：上传 CSV、TXT、JSONL 等文件，并记录文件 ID、格式和数量。
- 算子市场：查询 DataMate operator market，获取可用处理算子及其能力描述。
- 清洗任务：创建清洗任务并轮询状态，保留任务状态变化。
- 知识库检索：通过 DataMate 知识库检索支持后续问答和集成验证。

任务一智能体在调用 DataMate 前会做任务理解和数据 profile，在调用后会将 DataMate 返回的状态、结果和异常信息纳入 trace。这样既保留平台能力，也让智能体具备自主规划与异常处理能力。

6.2 Nexent 在系统中的位置

Nexent 是智能体平台入口。项目将三个智能体封装成 OpenAPI/MCP 工具并注册给 Nexent。Nexent 的作用包括：将智能体工具纳入统一工具列表、支持用户通过自然语言触发智能体、通过 OpenAPI/MCP 完成工具调用、保留 agent id、工具注册状态、工具可见性和运行记录。

任务	OpenAPI 服务	核心操作
任务一	modelengine_task1_data_processing_agent	run_datamate_processing_agent
任务二	modelengine_task2_medical_kg_agent	build_medical_kg_agent、ask_medical_kg_agent
任务三	modelengine_task3_analysis_agent	run_analysis_agent、run_interactive_analysis、run_configured_analysis

7 任务一：数据处理智能体详细设计

任务一要求基于 Nexent 构建数据处理智能体，并利用 DataMate 的数据处理能力。项目中的 DataProcessingAgent 面向医疗数据接入、清洗、抽取、转换和质量控制场景，支持结构化表和非结构化文本。

7.1 输入输出与可运行 Demo

任务一支持医疗文本 JSONL、结构化 CSV、普通 TXT、多文件批量输入和自然语言任务描述。输出包括标准化 documents.jsonl、数据质量报告、处理计划、执行调度表、DataMate 任务状态记录、异常

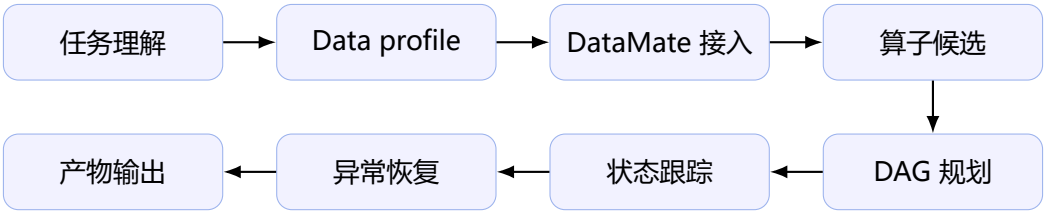


图 3: 任务一数据处理智能体链路。任务一可通过 CLI、HTTP API 和 Nexent OpenAPI/MCP 工具触发。

恢复报告和后续 KG 构建输入。

```
python3 -m demo.run_medical_demo process \
  --output-root . \
  --input data/raw/medical_demo.jsonl \
  --task "清洗医疗文本并抽取字段，标准化为图谱输入"
```

任务一 Demo 截图式说明

CLI 入口	python3 -m demo.run_medical_demo process ...
HTTP API	POST /api/task1/process
Nexent 工具	modelengine_task1_data_processing_agent_run_datamate_processing_agent
关键执行流	任务理解、Data profile、DataMate 接入、算子候选、DAG 规划、状态跟踪、异常恢复、产物输出
任务一输出	documents.jsonl、data_processing_trace.jsonl、data_quality_report.json、data_processing_planning.json

7.2 任务理解与语义约束图

DataProcessingAgent 使用语义约束图表示用户任务。语义约束图包含 primary intent、多意图列表、必须覆盖的能力、质量约束、推荐算子、规避算子和输入约束。例如用户输入“对批量医疗结构化表和非结构化文本做 ETL、清洗、字段抽取、质量校验、去重和异常恢复”，系统会同时识别 ingest、clean、extract、transform、deduplicate、validate、recover 等能力，并把多文件批处理作为规划约束。

7.3 多目标 DAG 规划

任务一不是按固定顺序执行所有算子，而是根据任务约束和数据 profile 搜索 DAG。DAG 搜索的目标包括能力覆盖最大化、质量保留、执行成本控制、算子兼容、异常恢复和 DataMate 能力优先。规划结果会写入 data_processing_planning.json，并在报告中展示关键步骤。

能力	实现说明	产物
结构化 ETL	识别 CSV 字段、缺失值、重复值和类型，生成标准化 JSONL	documents.jsonl、schema_profile.json
非结构化文本处理	清洗医疗文本、过滤空文本、保留证据字段	documents.jsonl、document_profile.json
异常恢复	记录空文本、重复、缺字段、格式错误和 DataMate 调用异常	task1_invalid_processing_report.md
状态跟踪	记录 DataMate 任务状态、DAG 层级、输入输出数量、耗时	data_processing_trace.jsonl

8 任务二：医疗知识图谱智能体详细设计

MedicalKGAgent 将任务一处理产物和 CBLUE 转换数据统一输入到医疗知识图谱构建流程。系统围绕疾病、症状、药物、检查、治疗、危险因素、科室、临床阶段等实体类型构建图谱，并支持问答。

8.1 医疗数据源

项目使用 CBLUE 相关数据作为主要医疗数据来源。CMeIE 用于医学实体关系抽取，CMeEE 用于医学实体识别，CMedCausal 用于医学因果关系补充，项目样例医疗文本用于端到端 Demo 和回归测试。转换脚本将不同数据集统一成 document、entity、triple 三类结构，每条三元组保留 subject、predicate、object、实体类型、source doc id、evidence 和 confidence。

8.2 图谱 schema

图谱实体类型包括 Disease、Symptom、Drug、Treatment、Test、Anatomy、Department、RiskFactor、ClinicalStage、PatientGroup、Epidemiology、TransmissionRoute 等。关系类型包括 has_symptom、treated_by、diagnosed_by、has_complication、has_risk_factor、prevented_by、causes、located_in、belongs_to_department、has_stage 等。

8.3 核心算子

任务二实现多个核心算子：实体识别算子、关系抽取算子、三元组生成算子、三元组校验算子、实体链接算子和 tensor 候选排序算子。校验算子会进行类型校验、关系校验、冲突过滤、证据聚合和置信度计算。问答模块基于知识图谱和 DataMate 知识库检索结果回答医疗问题，并保留引用三元组和证据来源。

模块	说明
实体识别算子	抽取 Disease、Symptom、Drug、Treatment、Test、RiskFactor 等实体
关系抽取算子	抽取诊断、治疗、并发、风险、预防和因果关系
三元组校验	校验 subject-predicate-object 类型约束、过滤冲突关系
问答智能体	基于图谱和知识库检索组织自然语言回答

9 任务三：数据分析智能体与可视化

InsightAnalysisAgent 负责从“知识”走向“洞察”。它读取任务一标准化文档和任务二知识图谱，自动理解图谱结构，并执行 NL2SQL、统计分析、关联分析、图谱子图抽取和 BI 图文展示。

9.1 图谱感知与 NL2SQL

分析智能体会读取 schema、entities 和 triples，识别实体类型、关系类型、关系方向和实体属性。项目采用 query catalog + schema linking + 安全 SQL 校验方式，将自然语言问题映射为可执行查询。该方案既支持自然语言入口，又保证 SQL 安全和结果稳定。

9.2 BI 与图文展示

分析结果会输出 Markdown 分析报告、JSON 查询结果、BI 图表 JSON、图谱子图 JSONL、visualization manifest 和离线 Dashboard。Dashboard 展示实体分布、关系分布、图谱预览、问答结果、NL2SQL 结果和平台联调状态。

分析类型	示例	输出
统计分析	疾病关联药物数量、症状频率、科室分布	表格、柱状图
关联分析	症状共现、疾病风险因素、疾病治疗关联	排行榜、子图
图算法分析	核心实体、节点中心性、关系密度	中心性列表、图谱子图
质量分析	低置信度关系、高置信度关系类型	质量明细、复核列表

10 模型侧增强：Qwen3-0.6B + LoRA 编排策略模型

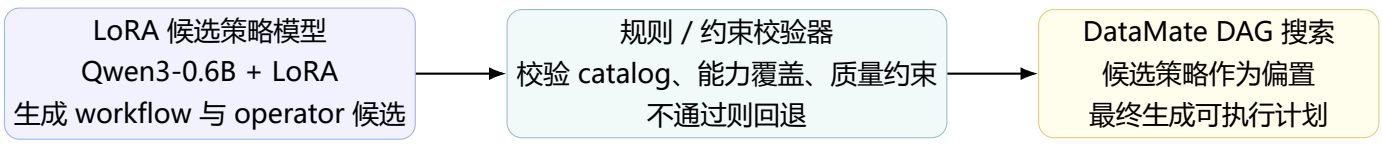


图 4: 模型侧增强接入方式。LoRA 生成候选策略，规则/约束规划器负责最终校验。

10.1 为什么需要模型侧增强

任务一的核心要求是让智能体具备任务理解、规划和自动执行能力。规则规划器可以保证稳定性，但面对复杂自然语言、多文件输入和多算子组合时，仅依赖规则容易出现覆盖不足或泛化不足。因此项目引入小模型编排策略增强，让模型学习“任务描述、数据 profile、能力需求、算子组合”之间的映射关系。

10.2 训练数据与训练方式

训练数据不是只靠少量模板生成。项目先从真实医疗样例文件、CBLUE 转换数据和结构化表样本中自动采样任务，再批量运行 DataMate/Nexent 链路采集反馈，随后引入 OOD 场景扩展和约束感知 DAG 校准。每条样本都包含任务文本、输入 profile、目标产物、候选 workflow、operator 列表、约束检查结果和最终策略标签。这样构造的数据能够让模型学习“用户意图 + 数据形态 + 算子能力 + 质量约束”之间的对应关系，而不是只记住单一固定流程。

训练过程采用 Qwen3-0.6B 作为基座模型，使用 LoRA 微调方式训练数据处理编排策略模型。选择 0.6B 级别模型的原因是它满足小于 1B 的规模要求，同时具备基础语言理解能力，适合作为任务理解和候选策略生成器。LoRA 只训练少量适配参数，便于分发、加载和复现，也便于后续继续用新的 DataMate 反馈数据增量更新。

项目	内容
基座模型	Qwen3-0.6B
微调方式	LoRA
训练样本	56,000 条数据处理编排指令样本
holdout/OOD operator micro F1	0.9943
manual blind v2 operator micro F1	0.9553
主链路角色	候选策略增强，不无保护接管执行链路

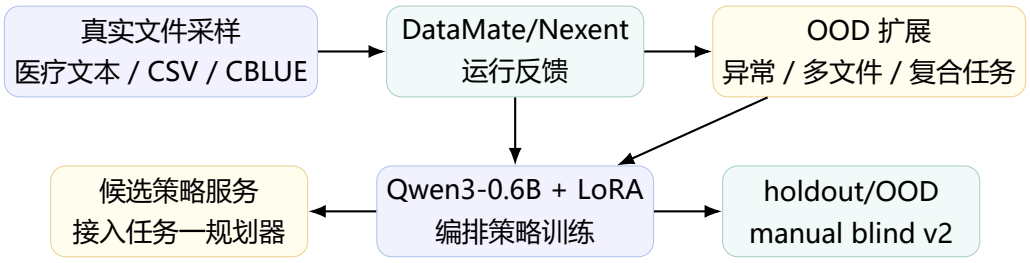


图 5: 小模型训练与接入链路。真实运行反馈和 OOD 扩展用于减少模板化数据带来的思维定式。

10.3 接入主链路

模型输出不会直接执行。主链路采用三阶段保护：先由 v7 LoRA 生成候选 workflow 与 operator 组合，再由 transformer lora guard 做结构、能力、约束和算子目录校验，校验通过后作为 DAG 搜索候选偏置；不通过则记录原因并回退稳定规划器。这种设计保留模型带来的泛化能力，同时避免模型幻觉直接影响 DataMate 任务执行。

在任务一中，模型主要增强三个环节。第一是复杂自然语言任务的意图拆解，例如同时包含清洗、抽取、转换、批处理和异常恢复的任务。第二是候选 operator 组合建议，例如在检测到结构化字段时优先建议 schema profile 和 field extractor，在检测到低质量文本时建议质量门禁与恢复动作。第三是 DAG 搜索启发式，让规划器更快找到覆盖率更高、质量约束更完整的计划。

11 NPU 算子优化重点说明

NPU 优化选择适合向量化、矩阵计算和 top-k 排序的真实算子，不把整个 Python 流程泛化宣称为 NPU 加速。当前覆盖任务一 ranker、任务二 KG tensor kernel 和任务三图谱相似度 tensor 算子。

任务	优化对象	当前结果
任务一	DataMate operator market 候选排序 tensor ranker	host CPU seconds/operator-score 改善 1.5921x
任务二	KG candidate ranking + top-k kernel	float32 核心 kernel 15.3177x，large batch 14.0649x
任务三	图谱多关系疾病相似度 tensor 算子	pair-score 吞吐约 10.1442x，p50 延迟约 10.0889x

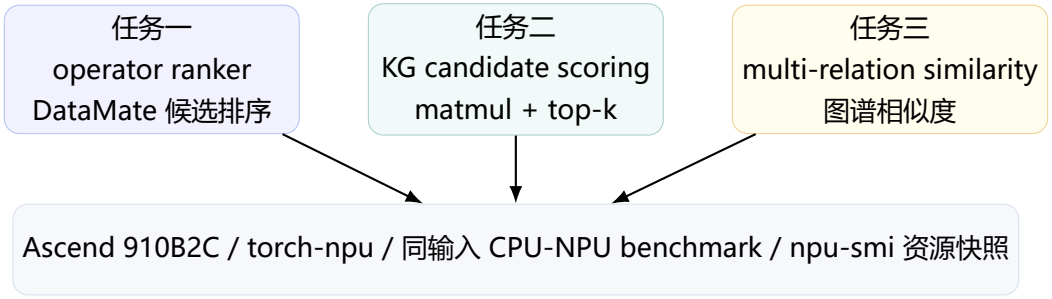


图 6: NPU 优化覆盖范围。优化对象是三任务中的真实 tensor 算子，而不是不可控的平台内部实现。

11.1 任务一 NPU Ranker

任务一的 NPU 优化对象是 operator planning ranker。智能体从 DataMate operator market 得到候选算子后，需要根据任务文本、能力需求、推荐算子、规避算子和数据 profile 对候选进行排序。该排序适合转化为 tensor 特征计算。当前结果表明 wall-clock 基本持平，主要收益是降低 host CPU 占用。

11.2 任务二 NPU KG Tensor Kernel

任务二是 NPU 优化重点。医疗 KG 构建过程中，需要进行实体候选召回、关系意图排序和 top-k 选择。该部分可以抽象为候选向量与查询向量之间的批量相似度计算，并使用 top-k 选择最相关候选。CPU 和 NPU 使用相同输入、相同 dtype、相同 vector dimension、相同 top-k 和相同 repeats。

口径	CPU scores/s	NPU scores/s	提升
float32 kernel	664,986,983.827	10,186,039,748.2617	15.3177x
bfloat16 kernel	3,058,408,102.0865	16,182,824,831.391	5.2913x
large batch float32	822,059,726.2344	11,562,222,602.0902	14.0649x

11.3 任务三 NPU 图谱相似度算子

任务三需要做图谱关联分析，其中一个典型任务是计算疾病之间的多关系相似度。例如两个疾病是否共享症状、药物、检查、风险因素和并发症。该任务可以转化为 multi-relation feature matrix 上的相似度计算。当前 pair-score 吞吐约 10.1442x，p50 延迟约 10.0889x。

NPU 优化边界

NPU 数字对应算子级 benchmark。项目不宣称 DataMate 官方内部清洗算子由本项目直接 NPU 加速，也不宣称三任务全流程端到端都达到同样加速倍数。

12 运行结果、交付方式与复现

项目	当前结果	项目	当前结果
文档数	320	实体数	4468
三元组数	3403	关系类型	15
KG Gold F1	97.24%	NL2SQL	70/70
DataMate retrieve	HTTP 200	Nexent agent run	三个任务均成功

12.1 可运行入口

```
bash scripts/prepare_submission.sh \
  --knowledge-base-id d769e9b3-2348-46d1-b0ab-bbaf499d748d \
  --cblue-source data/external/cblue \
  --cblue-limit 300 \
  --no-screenshots
```

分任务运行：

```
python3 -m demo.run_medical_demo process --output-root . --input data/raw/medical_demo.jsonl
python3 -m demo.run_medical_demo build --output-root .
python3 -m demo.run_medical_demo analyze --output-root .
python3 -m demo.run_medical_demo nl2sql-eval --output-root .
python3 -m demo.app_server --host 127.0.0.1 --port 8088
```

Dashboard 地址：http://127.0.0.1:8088/dashboard。

12.2 交付文件

类型	文件
项目书	docs/ModelEngine 初赛项目书终稿.tex、docs/ModelEngine 初赛项目书终稿.pdf
任务一	data/processed/documents.jsonl、data_processing_planning.json、demo/task1_processing_report.md
任务二	graph/entities.jsonl、graph/triples.jsonl、graph/kg_schema.json、demo/kg_quality_report.md
任务三	demo/analysis_report.md、demo/nl2sql_eval_report.md、demo/dashboard.html
集成	demo/integration/integration_evidence_report.md、platform_evidence.json
NPU	docs/npu_optimization_summary.md、docs/task2_tensor_kernel_1_5x_npu_report.md

模型包通过网盘提供：ModelEngine 初赛模型，链接 https://pan.baidu.com/s/1s-2L7JgXvSVuxSVV80hPCQ?pwd=5rxb，提取码 5rxb。最新 v7 LoRA adapter 包为 modelengine_task1_lora_qwen06b_v7_targeted.zip。

13 项目总结

ModelEngineBySoldierBoy 将 DataMate 数据处理能力、Nexent 智能体框架、医疗知识图谱、NL2SQL 数据分析、小模型编排策略和 Ascend NPU 算子优化组织成一个统一系统。项目不仅能跑通三项任务，还能展示数据在三个智能体之间如何被持续复用：任务一输出标准化数据，任务二将数据转化为知识，任务三再将知识转化为洞察和可视化结果。

项目的重点能力包括：DataMate 数据处理链路、Nexent OpenAPI/MCP 智能体接入、任务一多算子规划、任务二医疗语义抽取与校验、任务三 NL2SQL 与 BI 可视化、Qwen3-0.6B + LoRA 编排策略模型、Ascend NPU tensor 算子优化。整体系统既能运行，也能继续扩展到更多医疗数据源、更多数据处理算子、更多图谱分析场景和更强的本地小模型策略。